Golden Thread — Getting Started

Reader: the first thirty minutes — the happy path only **Claims last checked against the code:** 2026-10-02 (gt 0.19.1) — see *The documents, and what belongs in each* in `CLAUDE.md` .

A guided walkthrough for your first session. Six steps, ~15 minutes.

Requirements: Python 3.8+, Claude Code installed. Written against **gt v0.21.0**. The installer now asks its questions in order, security first (`bash install.sh --questions` lists them); answer them at the prompt or by flag. The last question can open the Getting started guide (`/gt:gt-help`).

Step 1 — Install

Unzip the plugin and run the installer:

```
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh --vault ~/Documents/GoldenThread
```

Then **restart Claude Code**. The `/gt:` skills are not available until you do.

`--vault` names the folder your memory will live in, and lets the installer finish the job: it creates the vault (or connects one you already have) and wires the Core-rule enforcement hooks, which are wired against a vault and cannot be wired without one.

Leave it off and the installer asks, if you are at a terminal. If something else is running it — an agent, CI — it stops with exit 4 and asks for the decision rather than choosing a directory for you. `bash install.sh --no-vault` installs the plugin alone, deliberately.

The installer sets up **`gt`** — memory, projects, session workflow, and enforcement hooks — plus each optional **module** that is on: - **`gt-wiki`** — knowledge base with immutable sources - **`gt-demo`** — the guided tour, in its own throwaway vault - **`gt-watch`** — follow git repos you depend on (`/gt-watch:gt-watch`) - **`gt-report-card`** — the session report card at `/compact` - **`gt-flow`** — a picture of how your knowledge moved (`/gt-flow:gt-flow`) - **`gt-visualize`** — a 3D walkthrough of how a codebase works, or a code city of it (`/gt-visualize:gt-visualize`) - **`gt-farm`** — work packets for an external AI; **off** unless you add it with `bash install.sh --with farm` (or had `/gt:gt-farm` before upgrading)

`bash install.sh --list-modules` shows each one and why it is on or off; `--without <name>` leaves one out, and the choice is remembered.

Step 2 — Initialize your vault

Open any Claude Code session and run:

```
/gt:gt-init
```

Claude will ask for: 1. A vault path — an Obsidian vault folder is ideal, but any folder works 2. Your domain/team name — e.g. "Platform Engineering" or "My Team"

The script creates the folder structure, writes `~/.claude/vault-config.json` (the pointer every other skill reads), and **installs Core-rule enforcement hooks** into `~/.claude/settings.json`. Safe to re-run on a machine you've already set up.

If you already have an Obsidian vault, just give `gt-init` that path. It will only add what's missing — your existing notes are untouched.

Looking at the vault in Obsidian (optional). A vault is a folder, so there is no import: install Obsidian, choose **Open folder as vault**, and pick this folder. The vault's own `OPEN-IN-OBSIDIAN.md` has the path and the details. Install **Dataview** if you want to sort tasks by the `[p:: 1]` inline fields, and **Obsidian Git** if you want your hand edits committed on a timer alongside Claude's.

Step 3 — Create your first project

```
/gt:gt-create
```

Claude will ask for a project name and a quick brain dump of what it is. That brain dump becomes `idea.md` — the immutable origin story for the project.

For a project you're already working on, use:

```
/gt:gt-ingest
```

This scans your existing notes and memory files and imports them into the vault without touching the originals.

Step 4 — Open a project at the start of each session

```
/gt:gt-open <project-name>
```

This reads your project docs in order — idea → research → decisions → design → spec → runbook — reads CONVENTIONS.md and PROTOCOL.md once per session, then reads the session memory index and stops. It summarizes where things stand and asks where to pick up.

Why it stops at the index: a project with 70 notes would otherwise dump 2,000 lines into context. `gt-open` reads the one-line index and loads individual files only when you need them. If `research.md` exceeds ~200 lines, it reads its one-line-per-entry digest (`research-digest.md` , kept current by `/gt:gt-work`) or, failing that, the headings, then only the relevant sections. Come back after a week away and it opens with a short generated catch-up first.

Step 5 — Write back at the end of each session

```
/gt:gt-work
```

This is the step that keeps the vault alive. Claude will write back what was learned:

DESTINATION	WHAT GOES THERE
`research.md`	New findings and gotchas, dated
`decisions.md`	Architectural choices (ADRs)
`design.md`	Updated architecture if it changed
`runbook.md`	Operational commands and landmines
`memory/*.md`	Session state

`gt-work` also checks whether any findings should be promoted wider (to `Knowledge/` or `global-memory/`) and asks you before writing. It updates the project's `README.md` frontmatter properties (`stage` , `topology`) if the project changed phase.

Skipping `gt-work` is how the vault decays. Facts that aren't written down are forgotten by the next session.

Step 6 — Capture from anywhere, then ask what's next

Steps 4 and 5 keep one project honest. This step is what keeps *all* of them in view, and it is the step that matters most if your attention is split across many things.

Capture wherever you are. You are working in project A and a thought about project B arrives. Do not switch projects. Add one checkbox line to `<vault>/INBOX.md` :

```
- [ ] Chrome extension: the confirm dialog retry may be resubmitting instead of confirming
```

No project, no priority, no date. The rollup shows every unchecked inbox line at the top of `TASKS.md`, so it cannot be lost, and nowhere else, so it does not rank. (If you keep Obsidian daily notes, the sweep reads those too.)

Sweep the inbox. Once a week, or whenever the notes pile up:

```
/gt:gt-review
```

Claude reads the inbox, asks where each line belongs, and files it: a task under an existing project, a sub-project, or a new project with its own `idea.md`. The inbox line is checked off with a `→ [[slug]]` pointer so the same thought is never surfaced twice. An idea becomes a `p:: 3` task with no due date.

Ask what's next. Every project keeps its open tasks under `## Tasks` in its `README.md`, one checkbox per task, with optional fields:

```
- [ ] Reproduce the dialog retry against a live page [p:: 1] [waiting:: agent] [since:: 202
```

One command rolls every project's tasks into a single ranked list:

```
python3 <vault>/Projects/golden-thread/tools/gt_tasks.py    # then read TASKS.md
```

Rank is computed against the clock, not stored: a `p:: 1` older than seven days escalates, a `due` inside three days escalates. **Regenerate before reading** — a stale `TASKS.md` is last week's ranking. `waiting:: user` is the list of things blocked on you; `waiting:: agent` is what the next session should pick up.

That is the loop: capture without switching, sweep to file, regenerate to decide.

Close what is finished. The rollup's *Review* section, the `/compact` report card (report-card module) and `gt-work` all ask the same question when a project's signals say it may be done: most tasks past due, most tasks checked, three quiet weeks, or nothing open. Answer yes, no or later; every answer is recorded so the question learns your pattern. Closing sets `stage: complete` and shelves leftover tasks at `p:: 7` — they stay in the README, they just stop outranking live work.

Typical session pattern

```
/gt:gt-open my-project     ← load context, see where you left off

# ... work happens ...

/gt:gt-route               ← lost? drifted? "where is this going, and where
                             does it belong?" Safe to run any number of times

/gt:gt-work                ← write back findings before closing
```

Periodic maintenance

```
/gt:gt-lint                ← find broken links, orphaned pages, unlisted memory files, unen
/gt:gt-promote             ← graduate a finding that proved true across projects
/gt:gt-refresh             ← check if any source documents changed upstream
/gt:gt-review              ← file the inbox (INBOX.md, plus daily notes if you keep them)
/gt:gt-settings            ← view and toggle what Golden Thread does automatically
/gt:gt-validate            ← independently verify a claim before recording it as fact

# from modules
/gt-watch:gt-watch         ← watch a git repo you depend on; a security fix upstream opens
/gt-flow:gt-flow           ← draw how knowledge climbed the ladder, one offline HTML file (
/gt-visualize:gt-visualize ← explain a codebase in 3D, or draw it as a code city (one offli
/gt-farm:gt-farm           ← route bulk or mechanical tasks to an external AI service as a
/gt-demo:gt-demo           ← guided tour in a throwaway demo vault, nine acts plus one per
```

Optional: require your presence (gt unlock)

gt unlock ships **off**, and the installer says so once. Turned on, the assistant needs you — TOTP from your phone plus Touch ID (macOS) or Windows Hello — before it uses LOTR connections, credentials, publishing or gt's own guards; ordinary vault work is never gated. In your own terminal:

```
U=~/.claude/golden-thread/hooks/gt_unlock.py
python3 $U enroll touchid      # macOS (Windows: hello; Linux: skip)
python3 $U enroll totp         # scan the QR code, type one code
python3 $U recovery            # ten one-time codes: store them offline
python3 $U policy enable       # proves the factors work, then turns it on
python3 $U verify              # the level this machine runs at, check by check
```

It is not anti-malware: something already running as you can wait for you to unlock. In 0.20.1 the gate is **friction** against the assistant and accidents, not a boundary: a program running as you — the assistant's own shell included — can get around it if it sets out to. What holds even then, with Touch ID / Hello (L2): every open of a sealed credential needs your finger or PIN, and so can each consent-tier operation. `SECURITY.md` says exactly what it stops and what it does not.

Optional: fence the assistant off the vault (gt sandbox mode)

gt sandbox mode ships **off**. Turned on, gt writes Claude Code's own sandbox settings and permission rules so the assistant's shell and file tools cannot write the vault or gt's state, or read your second factor and credentials; it reads the vault through gt's MCP tools and writes only through the write queue. In your own terminal:

```
S=~/.claude/golden-thread/hooks
python3 $S/gt_settings.py set sandbox_mode on    # Linux / WSL2: needs bubblewrap and socat
python3 $S/gt_sandbox.py status                  # what it enforces on this platform
# restart Claude Code; switch off the same way, from a terminal
```

On macOS, Linux and WSL2 the operating system enforces it around the assistant's shell. On native Windows Claude Code has no sandbox, so only the file-tool rules apply: **friction, not a boundary**. It does not stop malware running as you. `SECURITY.md` §8 lists exactly what it writes, what it stops and what it does not.

Adding a knowledge base (gt-wiki)

If you want a structured wiki alongside your project memory:

```
/gt-wiki:gt-wiki-init
```

Then add sources:

```
/gt-wiki:gt-wiki-ingest <url or file path>
```

Query it:

```
/gt-wiki:gt-wiki
```

Check its sources for upstream changes:

```
/gt-wiki:gt-wiki-refresh
```

The wiki uses immutable sources — every ingested document is stored verbatim and never modified. Knowledge pages are synthesized summaries that link to the originals. When a source changes upstream, refresh adds a new source that supersedes the old one; for sources in a local git repo the change is detected by `git diff`, not by eye.

Key concepts in one paragraph

The vault is organized by **how widely a fact applies**. Session findings go in `memory/`. Settled discoveries go in `research.md`. Choices that shaped the architecture go in `decisions.md`. Facts true across multiple projects go in `Knowledge/`. Facts needed in every session everywhere go in `global-memory/`. Above all of these sits the **Core-rule tier**: rules enforced via hooks wired into `~/.claude/settings.json`, not merely stored as files. `gt-init` installs these hooks automatically. Promote a fact when a second project proves it general — not before. When unsure, file it narrow; promotion is cheap, demotion isn't. One more principle: **claims should be verified by re-deriving them, not by reviewing the reasoning that produced them** — a reviewer who sees your reasoning inherits your blind spot. Use `/gt:gt-validate` before recording any finding that matters.

Troubleshooting

Skills not showing up — restart Claude Code after install.

"vault-config.json not found" — run `/gt:gt-init`.

A rule seems ignored — `CLAUDE.md` is read at session start; rules added mid-session apply from the next session.

Memory files aren't loading — that's by design. Ask for a file by name, or run `/gt:gt-lint` to check if it's unlisted.

Core rules seem to apply sometimes but not always — run `/gt:gt-lint` and look for `core-unenforced` findings. The hook may not be wired. Fix:

```
python3 <plugin>/scripts/vault_init.py install-core-rules --vault <vault>
```

I want to turn off the automatic session report — run `/gt:gt-settings set report_card off`, or remove the module with `bash install.sh --without report-card`.

Timestamp missing from responses — the enforcement hook is unwired. Run `/hooks` in Claude Code or restart. If it persists, re-run `install.sh`.

For the complete skill reference, see MANUAL.md.